

Polarity Tester using Arduino Nano and CAN Protocol

PRATYUSH MESHAM
IIT(BHU)

JUNE 2025

CERTIFICATE

This is to certify that the project entitled “**Polarity Tester using Arduino Nano and CAN Protocol**” submitted by Pratyush Meshram of Indian Institute of Technology (Banaras Hindu University), Varanasi, was carried out as part of the internship at **Inverted Energy Pvt. Ltd.**, is an original and authentic work. The project was undertaken as part of practical training and professional exposure in the domain of electronics design and embedded systems.

The work was completed under the guidance and supervision of the technical team at Inverted Energy Pvt. Ltd., and demonstrates the application of theoretical knowledge to real-world industrial needs, such as detecting reverse polarity in battery packs and communicating the status via CAN protocol.



Mr. Akash Tiwari
Manager (Electronics and Electrical Team)
Inverted Energy Pvt. Ltd.

DECLARATION

I hereby declare that the work presented in this project titled **“Polarity Tester using Arduino Nano and CAN Protocol”** is an original and authentic record of the work carried out by me during my internship at **Inverted Energy Pvt. Ltd.** This work was done as part of my academic engagement as a student of Electronics Engineering, Indian Institute of Technology (Banaras Hindu University), Varanasi. It does not contain any part of the work, which has been submitted for the award of any degree either in this Institute or in other University/Deemed University without proper citation.



Pratyush Meshram

Indian Institute of Technology (Banaras Hindu University)

Contents

TOPICS	PAGE NO.
Abstract	4
Components Used	5
Schematic Diagram	5
PCB Layout	6
Working Principle	7
Arduino Code Summary	8
Results and Conclusions	10
Applications	10
Future Scope	10
Appendix	11
Bibliography	11

Abstract

In high-voltage battery systems such as a **16S LiFePO₄ (LFP) battery pack** with a nominal voltage of **51.2V**, ensuring correct polarity during connection is critical. If the battery is connected in **reverse polarity**, whether due to **human error**, **mislabelling**, **loose wiring**, **connector confusion**, or **faulty terminals**, it can lead to **dangerous consequences** such as:

- Short circuits
- Damage to the Battery Management System (BMS)
- Spark hazards
- Permanent damage to downstream circuits
- Safety risks for the user

Traditionally, the polarity check is done **manually using a multimeter** or by observing color-coded wires and terminals. However, this approach is **time-consuming**, **prone to human error**, and **inefficient for repetitive testing**, especially in industrial or testing setups.

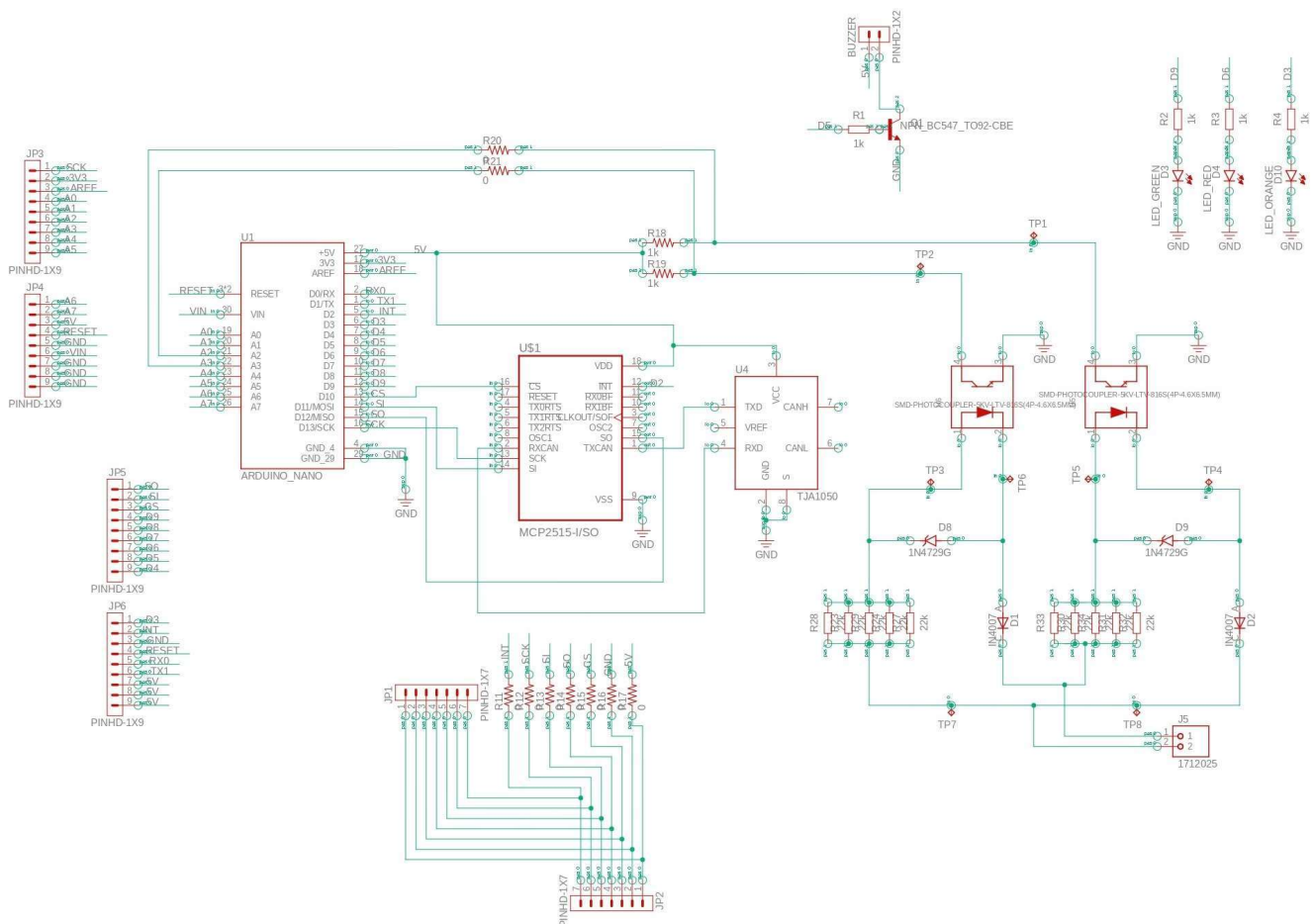
To overcome these limitations, an **automated polarity tester** was developed. This device takes the **51.2V input** from the battery pack and determines whether the connection is in **correct (positive)** polarity or **reverse** polarity. It uses **optocouplers** for safe voltage isolation and an **Arduino Nano-based control system** to identify the polarity. The detected polarity is then indicated using **LED indicators** and communicated over the **CAN protocol**, allowing safe and smart integration with other devices or test stations.

This solution eliminates manual dependency, reduces the risk of reverse connections, and provides **real-time polarity validation**, making battery installation and testing **safer, faster, and more reliable**.

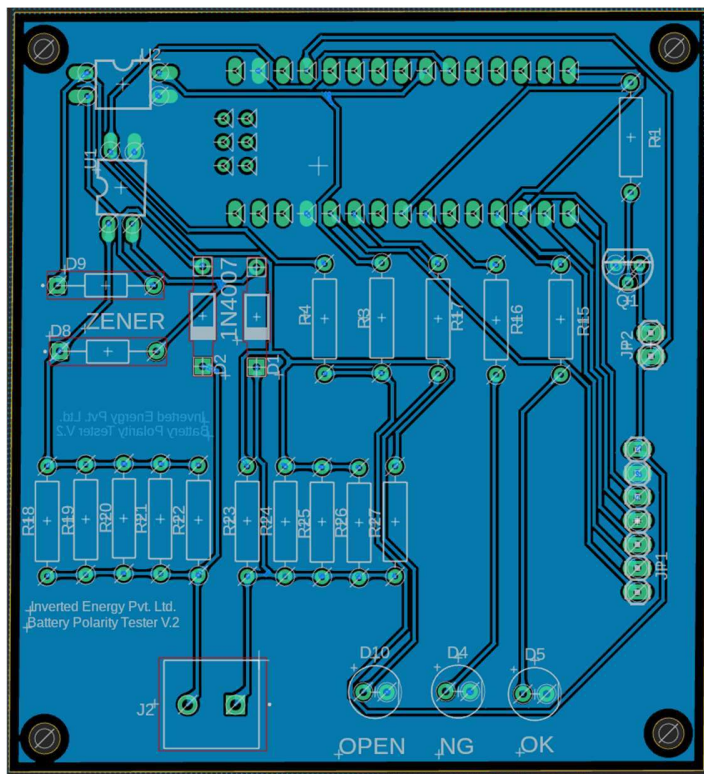
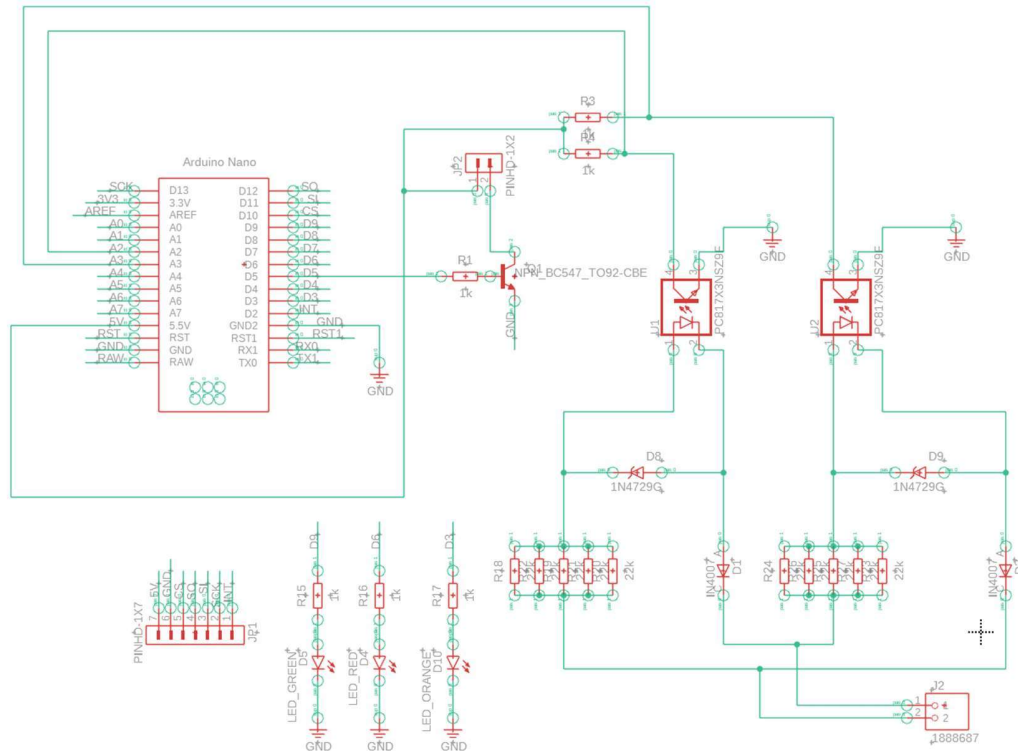
Components Used

- Arduino Nano (ATmega328P)
- MCP2515 CAN Controller Module
- TJA1050 CAN Transceiver
- PC817 Optocouplers (2 units)
- BC-547 (NPN Transistor)
- LEDs (3 units: Right Polarity (Green), Reverse Polarity (Red), No Polarity (Orange))
- Resistors: 1k Ω , 22k Ω
- Zener Diodes: 5.6V
- Diodes: 1N4729G
- 2-Pin Screw Terminal Block for Input
- Buzzer
- Zero PCB

Schematic Diagram



PCB Layout



The PCB layout was designed to reflect the complete schematic of the polarity tester circuit, incorporating all essential components such as the Arduino Nano, optocouplers, LEDs for polarity indication, a buzzer driver circuit, and supporting passive components. Particular attention was given to routing, spacing, and component placement to ensure reliability and manufacturability. The **trace width was set to 12 mil** after current and voltage calculations to ensure safe operation, and **teardrops** were added to improve solder joint strength and reduce mechanical stress at pad connections. Since no standard footprints were available for the MCP2515 CAN controller and the TJA1050 transceiver module, a set of 7 header pins was manually placed on the PCB. These headers were routed exactly according to the schematic, allowing the physical CAN module to be directly soldered onto the board with all its pins correctly connected. This approach ensures seamless functionality while offering a practical solution for integrating preassembled modules.

Working Principle

The polarity tester is designed to identify the correct or reversed connection of a high-voltage (51.2V) 16S LFP battery pack. The system uses an Arduino Nano in combination with optocouplers, indicator LEDs, a CAN controller, and a buzzer to detect and communicate the polarity condition safely and effectively.

1. Polarity Detection using Optocouplers

- Two optocouplers, **U1** and **U2**, are connected across the battery input to sense voltage polarity:
 - **U1** is activated when the battery is connected in the **correct polarity**.
 - **U2** is activated when the battery is connected in **reverse polarity**.
- These optocouplers isolate the high-voltage side from the low-voltage control circuitry, ensuring safety.

2. Signal Input to Arduino Nano

- The outputs of U1 and U2 are fed to the **analog input pins A2 and A3** of the Arduino Nano.
- The Arduino reads these analog signals using the `analogRead()` function to determine which optocoupler is conducting.

3. Logic and LED Indication

- Based on the input readings:
 - If **only U1 is active**, it indicates **right polarity**, and **LED D5** turns ON (Green LED).
 - If **only U2 is active**, it indicates **reverse polarity**, and **LED D4** turns ON (Red LED).
 - If **neither or both** are active (e.g., floating inputs or connection error), it is treated as **no valid input**, and **LED D10** turns ON (Orange LED).

4. Buzzer Alert for Reverse Polarity

- When **reverse polarity is detected**, in addition to turning on LED D4, a **buzzer** connected to **digital pin D5** of the Arduino (via a BC547 transistor switch circuit) is also **activated** to audibly alert the user of incorrect wiring.
- This ensures quick identification even in low-visibility conditions or noisy environments.

5. CAN Bus Communication

- The detected polarity status is encoded into a **CAN frame** and sent over the CAN bus using the **MCP2515 CAN controller**.
- The **TJA1050 transceiver** is used to drive the CANH and CANL lines, enabling communication over the physical CAN network.
- This allows the polarity status to be logged, monitored remotely, or trigger other actions in a larger Battery Management System (BMS).

6. Fail-safe Feature

- The system includes logic to ensure that **only one polarity** is reported at a time.
- If both optocouplers conduct simultaneously due to noise or fault, it defaults to **no input detection**, avoiding false positives.

Arduino Code Summary

- Polarity detected through analog pins A2 and A3

```
void loop() {
  int Value1 = analogRead(PolarityB_Pin); // A3 = opto U2
  int Value2 = analogRead(PolarityA_Pin); // A2 = opto U3

  if ((Value2 > 700) && (Value1 < 700)) {
    Value = Value2;
    Data = 0x65; // "Polarity B"
    Serial.println("Reverse Polarity B Detected");
    digitalWrite(LED_Pos, LOW);
    digitalWrite(LED_Neg, HIGH);
    digitalWrite(LED_Null, LOW);
    digitalWrite(BuzzerPin, HIGH); // Buzzer ON
  }
  else if ((Value1 > 700) && (Value2 < 700)) {
    Value = Value1;
    Data = 0x6F; // "Polarity A"
    Serial.println("Right Polarity A Detected");
    digitalWrite(LED_Pos, HIGH);
    digitalWrite(LED_Neg, LOW);
    digitalWrite(LED_Null, LOW);
    digitalWrite(BuzzerPin, LOW); // Buzzer OFF
  }
  else if ((Value1 > 700) && (Value2 > 700)) {
    Value = 0;
    Data = 0xFF; // "No Input"
    Serial.println("No Input Detected");
    digitalWrite(LED_Pos, LOW);
    digitalWrite(LED_Neg, LOW);
    digitalWrite(LED_Null, HIGH);
    digitalWrite(BuzzerPin, LOW); // Buzzer OFF
  }
}
```

- CAN Transmission

```

uint8_t CAN_Data[8] = {
    hexBytes[0], hexBytes[1], Data, 0x87, 0x82, 0x67, 0xAA, 0xCC
};

uint8_t dlc = 0x08;
uint32_t CAN_Id = 0x18784001;
CAN_Tx(CAN_Id, dlc, CAN_Data);

if (Serial.available() > 0) {
    uint8_t CAN_Data[8] = {0};
    uint32_t CAN_Id = 0;
    memset(UART_Read, 0, UART_SIZE);

    uint8_t Size = Serial.readBytes(UART_Read, UART_SIZE);
    if (Size == UART_SIZE) {
        uint8_t dlc = UART_Read[0] & 0x0F;
        ff = UART_Read[0] >> 7 & 0xFF;

        if (ff) {
            CAN_Id = ((uint32_t)UART_Read[1] << 24) |
                ((uint32_t)UART_Read[2] << 16) |
                (UART_Read[3] << 8) | UART_Read[4];
        } else {
            CAN_Id = (UART_Read[3] << 8) | UART_Read[4];
        }

        memcpy(CAN_Data, UART_Read + 5, dlc);
        CAN_Tx(CAN_Id, dlc, CAN_Data);
    }
}

uint8_t CAN_Tx(uint32_t id, uint8_t dlc, uint8_t *pData) {
    canMsgTx.can_id = id | CAN_EFF_FLAG;
    canMsgTx.can_dlc = dlc;
    memcpy(canMsgTx.data, pData, dlc);
    return !mcp2515.sendMessage(&canMsgTx);
}

void intToHex(int value, byte *hexBytes, size_t numBytes) {
    for (size_t i = 0; i < numBytes; i++) {
        hexBytes[i] = (value >> (8 * (numBytes - 1 - i))) & 0xFF;
    }
}

```

- After detecting polarity, the corresponding data (value and type) is formatted into a **CAN frame**.
- This frame is sent via the **MCP2515 CAN controller** and **TJA1050 driver** to communicate the result over the CAN bus.

Results and Conclusions

- **LED-Based Polarity Indication**

Three LEDs provide clear status:

- **Green** for Correct/Right polarity
- **Red** for Reverse polarity
- **Orange** for no or invalid input

This allows quick visual identification without tools.

- **CAN Communication for Remote Monitoring**

Polarity data is transmitted via the **MCP2515 CAN controller** and **TJA1050 transceiver**, enabling integration with battery management systems or other remote devices.

- **Onboard Test Points for Debugging**

Test points are included for key signals like optocoupler outputs and analog lines, making the circuit easier to debug and verify during testing or troubleshooting.

Applications

- Automotive polarity testing
- Industrial battery diagnostics
- Laboratory equipment verification
- Electrical maintenance and troubleshooting

Future Scope

- **Wireless Data Transmission:**

Upgrade the system using ESP32 or BLE modules to transmit polarity status wirelessly to a central monitoring system.

- **Automatic Shutdown/Relay Control:**

Implement a relay-based cutoff system that disconnects the load if reverse polarity is detected to enhance safety.

- **Multi-Channel Testing Support:**

Expand the design to check multiple battery channels or packs simultaneously for industrial setups.

Appendix

Bill of Materials (BoM)

Category	Component Description	Package	Quantity
Microcontroller	Arduino NANO (2.54mm pitch)		1
CAN Module	MCP2515 CAN Bus Module with TJA1050 Transceiver		1
Transistor	BC-547	TO-92	1
Optocouplers	PC817	DIP-4	2
Zener Diodes	1N4729 1W 3.6V	DO-41-2	2
Signal Diodes	1N4007	DO-41	2
LED Indicators	5mm Red LED	THT	1
LED Indicators	5mm Orange LED	THT	1
LED Indicators	5mm Green LED	THT	1
Resistors	1k Ω (0.25 W)	THT	6
Resistors	22k Ω (0.25 W)	THT	10
Connectors	2-pin headers (2.54mm pitch)	THT	1
Connectors	7-pin headers (2.54mm pitch)	THT	1
Input Terminals	2-pin screw terminals	THT	1

Bill of Materials (BoM) for the final PCB layout after successfully implementing and soldering the circuit onto a zero PCB.

Bibliography

- <https://forum.arduino.cc/t/arduino-polarity-detection/1269424>
- <https://forum.arduino.cc/t/can-bus-communication-between-tja1050-and-mcp2515-using-mcp2551/1177227>
- <https://www.homemade-circuits.com/optocouplers-working-characteristics-interfacing-application-circuits>
- <https://ww1.microchip.com/downloads/en/DeviceDoc/MCP2515-Stand-Alone-CAN-Controller-with-SPI-20001801J.pdf>
- https://wiki.seeedstudio.com/CAN-BUS_Shield_V2.0
- <https://github.com/autowp/arduino-mcp2515>